

Krav på program och dokumentation

Detta dokument handlar om hur ni *ska* skriva och dokumentera era program på kursen Datakommunikation I, och hur ni *borde* göra även annars.

Avsikten med dessa krav är att era program ska fungera, och i de fall de inte gör det (t.ex beroende på yttre faktorer) upptäcka att något gått fel, och vad. Avsikten är också att de program ni skrivit ska vara begripliga.

På de flesta kurser är uppgifter sällan större än att en ensam person kan överblicka hela problemet, men i industrin kommer man oftast att bara få jobba med en mycket liten del av system som ska underhållas i decennier. Då är det viktigt att man varit tydlig och dokumenterat väl.

Tyvärr sätts ofta likhetstecken mellan "mycket dokumentation" och "bra dokumentation". Bra variabel- och funktionsnamn räcker ofta långt, och ger ofta mer än flera A4-sidor med förklaringar.

God programmeringssed

Vad menas med "god programmeringssed"? Det är det något svårfångade begrepp som skiljer bra och dåligt skrivna program. De allra flesta icke-triviala program har **buggar**, skillnaden är oftast bara hur många och hur allvarliga. Skillnaden mellan väl och dåligt skrivna program är att den senare typen ofta inte ens märker att något går fel, och sällan gör något för att undvika ens uppenbara problem. Programspråket C och operativsystemet UNIX hjälper en tyvärr inte heller särskilt mycket, men ofta finns det sätt att begränsa skadorna.

Här följer några ganska rimliga krav på era program. Om ni försöker följa dem under *hela* utvecklingen av era program (dvs inte först när allt funkar), kommer ni eliminera många problem under utvecklingen.

- Returvärden från anrop till standardbibliotek och operativsystem som indikerar fel ska upptäckas och hanteras på rimligt sätt.

Exempel:

- De flesta I/O funktioner returnerar något värde som kan indikera fel. (Ofta ett negativt värde, men slå upp det i dokumentationen.) Det exakta felet finns i variabeln `errno`, och genom att inkludera `<errno.h>` kan man testa vilket fel som uppstod. Funktionen `perror()` kan användas för att få standardiserade felmeddelanden.

- Många I/O-funktioner talar om hur mycket de lyckats läsa eller skriva, vilket man bör utnyttja för att säkerställa att man lyckats läsa eller skriva så mycket som man förväntade sig.

Om man väntade sig 100 bytes, men bara fick 10, kan man misstänka att något inte står rätt till.

- Kom ihåg [Ariane 5, flight 501](#)! Felet som orsakade kraschen upptäcktes av systemet, men för att "optimera" hade man beslutat att inte införa kod som hanterade detta fel. Resultatet var att raketens styrsystem försökte använda felmeddelandet som flygdata...

- Strängar, buffrar osv får ej användas på ett sätt som gör det möjligt att överskrida det allokerade utrymmet, vare sig vid läsning eller skrivning.

Exempel:

- Strängar i C är lömska! Antag att du skapar en sträng på följande vis:

```
char string[80];
```

Hur många tecken kan då din sträng innehålla?

Svar: 79 tecken.

Detta för att sista tecknet i en sträng måste vara `'\0'`, som en indikation på att strängen är slut. Saknas denna, kommer sträng-funktioner fortsätta att läsa, eller ännu värre skriva, i minnet bortom slutet av det utrymme du avsatt. Detta är en mycket vanlig källa till oförklarliga fel, krascher, söndertrasad data och säkerhetshål.

Kom dessutom ihåg att strängar i C börjar på position 0, dvs `string[0]` är första positionen. Detta betyder att den sista positionen i den sträng vi deklarerat ovan är `string[79]` och **inte** `string[80]` som man kan tro när man ser deklarationen.

- Eftersom ett anrop till funktionen `gets()` kan skriva hur mycket data som helst till den sträng den tar som argument, bör funktionen aldrig någonsin användas. Hittar du en funktion med detta beteende, leta efter en alternativ funktion som har begränsningar.
Lösning: Använd `fgets()` istället.

- Funktionen `strncpy()` kopierar max N tecken, vilket gör det möjligt att säkerställa att oallokerat utrymme inte skrivs över. Då exakt N tecken kopieras, kommer dock inte något sträng-avslutande noll-tecken, `'\0'`, att skrivas. Om man senare använder strängfunktioner på denna sträng kommer data att läsas i oallokerat utrymme.
Lösning: Kopiera ett tecken mindre, och skriv alltid en `'\0'` i strängens sista position.
Exempelkod:

```
#define BUFLen 80

main() {
    char buf1[BUFLen*2], buf2[BUFLen];
    /* Kod som modifierar buf1 */

    strncpy(buf2, buf1, BUFLen - 1);
    buf2[BUFLen - 1] = '\0';

    /* Resten av programmet */
}
```

Notera dock att det troligen är något annat fel ifall det av dig anvisade buffer-utrymmet inte räcker till. I programmet ovan är det ju t.ex uppenbart att den kan bli problem, eftersom `buf2` är hälften så stor som `buf1`.

På strängar som systemet ger en kan det vara en bra idé att testa längden med `strlen()` innan man provar att kopiera dem.

Er enda chans att få godkänt för ett program som bryter mot detta, är om ni tydligt och mycket noga dokumenterar:

- Att problemet finns, och exakt var.
- Varför det uppstår.
- Varför det inte kan undvikas eller lösas.

Dokumentation av program

Era program ska dokumenteras såväl internt som externt. Som ett minimum ska er interna dokumentation bestå av:

- Vettiga variabel- och funktionsnamn.
 - Varje variabel ska ha ett namn som återspeglar vad den *används till*. Om något är en buffert, kalla den `buffert`, har du flera buffrar, tala då i namnet om vad det är för typ av buffert.
 - Återanvänd inte variabler, skapa två olika istället. Det krävs ofta snudd på tankeläsning för att inse exakt var en variabel som återanvänds byter betydelse i programmet. Byter den betydelse, borde namnet återspegla detta.
(Var å andra sidan inte rädd för att fortsätta använda en variabel som har rätt innehåll och namn. Att använda samma resultat-variabel för flera I/O-operationer är t.ex helt OK, så länge namnet och dokumentationen speglar detta.)
 - Funktioner ska ha namn som talar om vad de gör, och gärna på vad de gör det. Om du t.ex i ett paket för att hantera databasfunktioner har en funktion som hämtar data så är namnet `florp()` dåligt, `get_data()` lite otydligt och `db_get_data()` bra.
- För varje funktion, ett huvud som talar om vad funktionen gör, förväntad indata, förväntad utdata (inklusive ändringar av globala variabler och data refererad till av indata), och om funktionen inte är trivial, hur den löser uppgiften. Exempel:

```
/* int quicksort(list *namelist)
 * Funktionen sorterar en lista i bokstavsordning (A-Z)
 * Indata: En pekare till en lista (namelist) av strängar.
 * Utdata: Talet 1 om sorteringen lyckades, och talet 0 annars.
 * Dessutom förändras den lista som namelist pekar på.
 * Implementation:
 * Algoritmen från kapitel 9 i "Algoritms in C" av Sedgewick
 * har använts.
 */
int quicksort(list *namelist) {
    /* Kodan bortplockad */
}
```

- Alla viktigare delar av ert program bör dokumenteras. Det betyder svar på frågor som:
 - För varje global variabel: Vad den är till för, varför den är global och inte lokal, och i vilka funktioner den används.
OBS: Denna uppgift kan utan problem lösas utan globala variabler.
 - I huvudet på en loop: Varför vi loopar, och hur länge.
Exempel:


```
/* Loop until we get acknowledge for last packet */

while (! got_ack) {
    /* loopen */
}
```
 - I huvudet på en `switch()`-sats: Vad vi tittar efter.
 - Datatyper: Vad som är vad, och vad de ska användas till.
Exempel: Se filen `message.h` där formatet på ett meddelande definieras.
- Om ni gör något intrikat, avancerat eller ointuitivt. Bra program gör så lite såna saker som möjligt.
- Kom ihåg att det sällan kostar något att vara tydlig i ett program. En extra rad eller några extra

mellanslag kan ofta göra underverk för ett programs läslighet.

`b[s+o*8]` och `buffert[start + (offset * 8)]` betyder samma sak och kompilerar till samma kod. Vilken är enklast att läsa och förstå?

I princip ska det utifrån den interna dokumentationen kunna lista ut hur programmet fungerar. Man får givetvis anta att den som läser programmet är ganska van vid C-programmering och UNIX, men spåkula ska inte behövas såvida inte en bifogas programmet.

Extern dokumentation

Den externa dokumentationen ska ge en överblick över programmet, identifiera delarna och beskriva dem. Om ni är vana C-programmerare kommer ni att dela upp programmen i flera filer, och kanske låta klient och server dela på dem. Med en sådan uppdelning blir det lättare att identifiera delarna, eftersom det betyder att ni ofta bara behöver hänvisa till relevant fil.

Den externa dokumentationen ska innehålla:

- En övergripande beskrivning av programmens struktur, komponenter och hur de samverkar.
- Beskrivningar av alla viktiga funktioner. Ev. hjälpfunktioner behöver inte dokumenteras här. Av särskilt intresse är gränssnitt mellan olika delar av programmen.
- Datatyper. Vilka som finns, och vad de används till.
- Globala variabler. Vilka som finns, vad de används till, varför de måste vara globala, vilka funktioner som använder dem.

Med endast den externa dokumentationen ska det gå att förstå hur ert program är upplagt, hur det sitter ihop och hur det är tänkt att fungera. I princip ska inga kunskaper om C krävas av läsaren, även om man kan utgå från viss programmeringsvana.

[Back to the main page](#)